

Gradient Lens

Project Master Guide — MVP Blueprint

See exactly why a weight changed.

1. What Are We Building?

Gradient Lens is an interactive neural-network debugger for learning backpropagation. It is deliberately NOT a general neural-network playground or a prettier TensorFlow Playground.

The core idea is simple: a learner can click an individual weight, inspect its real value and gradient, see the numerical chain-rule calculation that produced that gradient, predict whether the weight should increase or decrease, reveal the answer, apply the update, and observe the effect on the network.

2. The Problem We Are Solving

Students often understand the sentence “backpropagation uses the chain rule” without being able to calculate the gradient of one specific weight on a concrete network.

Existing tools can show training, decision boundaries, or forward inference in detail, while code-based resources can show backpropagation mathematically. Gradient Lens combines these ideas into one focused interaction: inspect one parameter and reconstruct, from real numbers, why it changed.

3. Target Users

- Undergraduate CS/IT students who know the basic idea of neural networks but struggle to execute backpropagation by hand.
- Self-taught learners who understand conceptual explanations but want an interactive way to connect the formulas to actual numbers.

4. The Core Product Thesis

A learner should not merely see that a neural network trained. They should be able to select a single parameter and reconstruct, from real numbers, exactly why it changed.

Every feature should be judged against this thesis. If a feature does not strengthen individual-parameter understanding, it belongs in a later version or outside the project.

5. The MVP — Exactly What We Build

The first version is intentionally small:

- Fixed $2 \rightarrow 2 \rightarrow 1$ neural network
- XOR dataset only
- Sigmoid activation only
- Binary cross-entropy loss
- From-scratch scalar autodiff using Value nodes
- Forward propagation, step by step
- Loss calculation
- Backpropagation, step by step
- Click/select a weight
- Gradient inspector
- Numeric chain-rule expansion

- Predict → Reveal interaction
- Gradient-descent weight update shown explicitly
- Simple loss chart

The MVP is one interactive workspace, not a collection of pages.

6. The Main User Journey

Open Gradient Lens

- See the 2→2→1 network and current XOR example
- Step through the forward pass
- Compute prediction and loss
- Start backward pass
- Select a weight
- Inspect its value and gradient
- Predict whether the gradient is positive or negative
- Reveal the real chain-rule calculation
- Apply the gradient-descent update
- Re-run the forward pass
- Observe the new prediction and loss
- Repeat as desired

7. Why XOR?

XOR is a compact, hand-verifiable example where a hidden layer is necessary because the problem is not linearly separable. It has only four input/label combinations, making it practical to inspect individual calculations and verify the engine independently.

8. Mathematical Model

Neuron weighted sum:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$

Sigmoid:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Sigmoid derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Output:

$$\hat{y} = \sigma(z_{\text{out}})$$

Binary cross-entropy:

$$L = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

Output-layer simplification:

$$\partial L / \partial z_{\text{out}} = \hat{y} - y$$

Output weight:

$$\partial L / \partial w_{oj} = (\hat{y} - y) a_j$$

Hidden-layer weight:

$$\partial L / \partial w_{ji} = (\hat{y} - y) w_{oj} \sigma'(z_j) x_i$$

The hidden-layer expression is especially important because it gives the learner a concrete multi-step chain-rule calculation to inspect.

9. The Autodiff Engine

The engine is built around a scalar Value node.

Each Value contains:

- data — its current numerical value
- grad — accumulated derivative of the loss with respect to that value
- op — the operation that created it
- parents — the values used to create it
- _backward — the local derivative logic used during reverse-mode autodiff
- label — a display name such as `w_o1`, `z_h1`, or `a_h1`

The graph is topologically ordered, the loss gradient is seeded at 1, and nodes are processed in reverse order. Gradients must accumulate with `+=` rather than overwrite existing contributions.

10. Verification — Non-Negotiable

Before trusting the visualization, the mathematical engine must be verified.

1. Unit tests for addition, multiplication, sigmoid and BCE.
2. Gradient-accumulation test.
3. One complete hand-calculated $2 \rightarrow 2 \rightarrow 1$ example as a permanent regression test.
4. Numerical finite-difference gradient checking.
5. Differential checking against PyTorch autograd.
6. End-to-end XOR training test to verify that loss trends downward.

PyTorch is a verifier/test oracle, not the engine that powers the shipped product.

11. Technology Stack

Frontend only:

- React
- TypeScript
- Vite
- Tailwind CSS
- Canvas for the network visualization
- SVG for the loss chart
- Plain React state/context initially
- No backend
- No database
- No user accounts

The engine is intentionally written from scratch in TypeScript so the UI can expose the actual computation graph.

12. Project Structure

`src/`

`engine/`

value.ts
ops.ts
loss.ts
network.ts
backward.ts
optimizer.ts
__tests__/
dataset/
xor.ts
visualization/
layoutNetwork.ts
colorScale.ts
components/
NetworkCanvas.tsx
InspectorPanel.tsx
StepControls.tsx
PredictPrompt.tsx
LossChart.tsx
state/
useEngineStore.ts
App.tsx

13. Signature Feature — Gradient Inspector

When the learner selects a weight, the inspector should show:

1. Weight label
2. Current weight value
3. Gradient, once backward propagation has actually run
4. Symbolic chain-rule path
5. Same expression with real numbers substituted
6. Final gradient value
7. Prediction prompt
8. Correct/incorrect prediction result
9. Old weight → updated weight after gradient descent

The displayed numbers must always come from the real engine state. No fake demo values, scripted gradients, or estimated numbers.

14. Learning Design

The project uses active interaction rather than passive animation.

- The first inspected example can be fully worked to establish the pattern.
- Subsequent weight inspections use Predict → Reveal.
- The learner should connect gradient sign/magnitude to the direction and size of the weight update.

- Scaffolding should fade rather than permanently revealing every answer.

The MVP is not a course platform, LMS, quiz bank, or lesson-management system.

15. What We Are NOT Building in the MVP

Do NOT add these during MVP:

- Configurable architecture
- Multiple datasets
- ReLU/tanh
- Decision-boundary visualization
- Diagnostic/broken-network mode
- Vanishing-gradient demonstrations
- Dead-ReLU demonstrations
- MNIST
- User accounts
- Backend/database
- Chatbot tutor
- CNN/RNN/Transformer support
- Full lesson platform
- User-facing PyTorch verification

These are intentionally deferred or excluded to protect the core project.

16. V2 — If MVP Works

After the core engine and inspector are proven:

- Small configurable architectures
- Additional toy datasets
- ReLU and tanh
- Decision boundary
- Learning-rate/activation comparisons
- Diagnostic mode: “Why isn't this network learning?”
- Zero initialization / high learning rate / low learning rate scenarios
- Vanishing-gradient and dead-ReLU demonstrations where mathematically appropriate

17. V3 — Optional Advanced Work

Only if V2 is complete and there is genuine remaining time:

- User-facing PyTorch/TensorFlow.js verification
- Deeper networks
- Mini-batch visualization
- Small MNIST exploration
- Formal learner evaluation / pre-post testing

18. Development Order

1. Create Vite + React + TypeScript project and test setup.

2. Implement Value.
3. Implement basic operations.
4. Write operation tests.
5. Build fixed $2 \rightarrow 2 \rightarrow 1$ computation graph.
6. Implement topological backward pass.
7. Verify forward values.
8. Implement XOR forward pass.
9. Implement BCE.
10. Implement gradient descent update.
11. Run XOR training without UI.
12. Cross-check gradients against PyTorch.
13. Render static network.
14. Add forward step controls.
15. Add backward step controls.
16. Build gradient inspector.
17. Add Predict $\rightarrow$ Reveal.
18. Add update visualization and loss chart.
19. Perform an outside-peer walkthrough.
20. Only then decide whether V2 is justified.

19. Reliability Rules

- Never display a number that has not actually been computed.
- Use epsilon-based comparisons for floating-point tests.
- Clamp BCE inputs away from exact 0 and 1 for numerical stability.
- Detect NaN/Infinity and surface a clear unstable-training state.
- Validate editable numeric inputs.
- Keep engine correctness separate from visualization logic.
- When something fails, reset clearly instead of rendering misleading values.

20. What Will Make This Project Stand Out

The project should stand out through technical honesty and educational depth rather than feature count.

The strongest evidence will be:

- A real from-scratch reverse-mode autodiff engine.
- Correctness demonstrated through independent tests.
- Real numerical chain-rule explanations generated from the computation graph.
- A UI that connects mathematics $\rightarrow$ prediction $\rightarrow$ update $\rightarrow$ consequence.
- A sharply defined problem rather than a generic “AI visualization” claim.
- Clear documentation of design decisions and limitations.

The project should be presented as an interactive neural-network debugger for learning, not as a replacement for a full ML framework.

21. One-Sentence Scope Rule

Does this feature help a learner select a single parameter and reconstruct, from real numbers, exactly why it changed — while keeping the network small enough to understand?

If yes, consider it for the current version. If no, defer it or leave it out.

Quick Reference

Item	Decision
Project	Gradient Lens
Core network	Fixed $2 \rightarrow 2 \rightarrow 1$
Dataset	XOR
Activation	Sigmoid
Loss	Binary cross-entropy
Core engine	From-scratch scalar reverse-mode autodiff
Core interaction	Inspect $\rightarrow$ Predict $\rightarrow$ Reveal $\rightarrow$ Update $\rightarrow$ Observe
Frontend	React + TypeScript + Vite + Tailwind
Visualization	Canvas + SVG
Backend	None for MVP
Database	None for MVP
Verification	Unit tests + hand calculation + finite difference + PyTorch

MVP principle: Engine first. Verify the math. Then build the interface.